

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Программирование»

ОТЧЕТ
к лабораторной работе №7
на тему:
«ПЕРЕГРУЗКА ОПЕРАТОРОВ»
БГУИР 6-05-0612-01 18

Выполнил студент группы 553504
СТЕЦКО Ярослав Юрьевич

(дата, подпись студента)

Проверил ассистент каф. Информатики
РОМАНЮК Максим Валерьевич

(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Задание 1. Вариант 8. Класс – множество символов. В качестве поля – массив символов. Добавить метод, проверяющий корректность множества (нет повторений) – нужен сеттеру. Перегрузить +, -, * (пересечение). Сравнить на == и !=. (d) если множество пустое, оно false. Преобразовать в String – неявно, назад – явно.

Для заданной задачи реализовать следующее:

- спроектировать класс согласно варианту индивидуального задания.

Для класса использовать отдельный модуль;

- спроектировать конструкторы и свойства с контролем корректности вводимых значений;
- перегрузить метод toString();
- перегрузить математические операции (имеющие смысл для объектов класса), инкремент и декремент (изменить поля на 1), отношения (==, !=, <, >), true и false, преобразования типа;
- создать несколько объектов класса. Продемонстрировать использование конструкторов и свойств;
- продемонстрировать работу всех методов и операций.

2 ВЫПОЛНЕНИЕ РАБОТЫ

Для выполнения задания был создан класс CharSet, реализующий работу с множествами символов. В качестве внутреннего представления множества используется массив символов типа char[]. Для обеспечения корректности данных в классе реализована проверка на отсутствие повторяющихся элементов, так как множество не может содержать одинаковые значения. В классе были реализованы свойства, индексатор, конструкторы и методы, обеспечивающие удобную работу с объектами. Особое внимание уделено проверке корректности входных данных: при попытке создать множество с повторяющимися символами или передать пустую ссылку null генерируется исключение.

```
using System;

class CharSet
{
    private char[] elements = new char[0];
```

```

public char[] Elements
{
    get => elements;
    set
    {
        if (value == null)
            throw new ArgumentNullException("Массив не должен быть
null");
        if (!IsValid(value))
            throw new ArgumentException("Множество содержит
повторяющиеся элементы");
        elements = new char[value.Length];
        Array.Copy(value, elements, value.Length);
    }
}

public CharSet(char[] elements)
{
    Elements = elements;
}

public CharSet()
{
    elements = new char[0];
}

private static bool IsValid(char[] arr)
{
    for (int i = 0; i < arr.Length; i++)
        for (int j = i + 1; j < arr.Length; j++)
            if (arr[i] == arr[j])
                return false;
    return true;
}

public int Count => elements.Length;

public override string ToString()
{
    if (elements.Length == 0)
        return "{ }";
    string result = "{ ";
    for (int i = 0; i < elements.Length; i++)
    {
        result += elements[i];
        if (i < elements.Length - 1)
            result += ", ";
    }
    result += " }";
    return result;
}

public char this[int index]
{
    get
    {
        if (index < 0 || index >= elements.Length)
            throw new IndexOutOfRangeException();
        return elements[index];
    }
}

```

```

    }

    public static CharSet operator +(CharSet a, CharSet b)
    {
        ...
    }

    public static CharSet operator -(CharSet a, CharSet b)
    {
        ...
    }

    public static CharSet operator *(CharSet a, CharSet b)
    {
        ...
    }
}

```

Продemonстрируем работу класса CharSet в классе Program, используя все реализованные методы и операции. В программе производится ввод двух множеств символов, после чего выполняются операции объединения, пересечения, разности, сравнения и преобразования типов.

```

using System;
class Program
{
    static void Main()
    {
        CharSet A = ReadSet("A");
        CharSet B = ReadSet("B");
        Console.WriteLine($"A      +      B      =      {A      +      B}");
        Console.WriteLine($"A      -      B      =      {A      -      B}");
        Console.WriteLine($"A      *      B      =      {A      *      B}");

        Console.WriteLine($"A      ==      B      :      {A      ==      B}");
        Console.WriteLine($"A      !=      B      :      {A      !=      B}");

        string      str      =      A;
        Console.WriteLine($"CharSet      ->      string:      {str}");
    }
}

```

Результат работы программы продемонстрирован ниже (см. рисунок 1).

```

Введите символы для множества A (например: abcd): 435
Введите символы для множества B (например: abcd): 690

-- Свойства --
A = { 4, 3, 5 } (кол-во элементов: 3)
B = { 6, 9, 0 } (кол-во элементов: 3)

-- Индексатор --
A[0] = 4
A[1] = 3

-- Математические операции --
A + B (объединение) = { 4, 3, 5, 6, 9, 0 }
A - B (разность)    = { 4, 3, 5 }
A * B (пересечение) = { }

-- Инкремент и декремент --
A до ++: { 4, 3, 5 }
A после ++: { 5, 4, 6 }
A после --: { 4, 3, 5 }

-- Сравнение --
A == B : False
A != B : True
A > B (по кол-ву): False
A < B (по кол-ву): False

-- Операторы true / false --
Множество A: не пустое (true)
Пустое множество: пустое (false) – верно

-- Преобразования типа --
Неявное CharSet -> string: "435"
Явное (CharSet)"world" -> { w, o, r, l, d }

```

Рисунок 1 – Результат работы программы

ВЫВОД

В ходе лабораторной работы были изучены механизмы реализации полиморфизма в C# на примере создания специализированного типа данных, обеспечивающий строгий контроль корректности операций с дробями, автоматическую нормализацию, преобразование типов и поддержку расширенного функционала через перегрузку операторов математических действий, сравнения и преобразования.